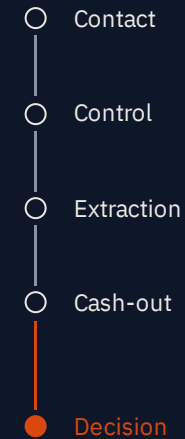


Chakravyuh

Intercept the scam workflow,
not the transaction.

Chakravyuh checks a UPI payment for the steps that usually come before a scam payment: a suspicious call, someone controlling the phone, an unusual payment, and a receiving account that looks like a mule. It interrupts only when several of these show up together, tells the customer why, and keeps a signed record for the bank.



Team

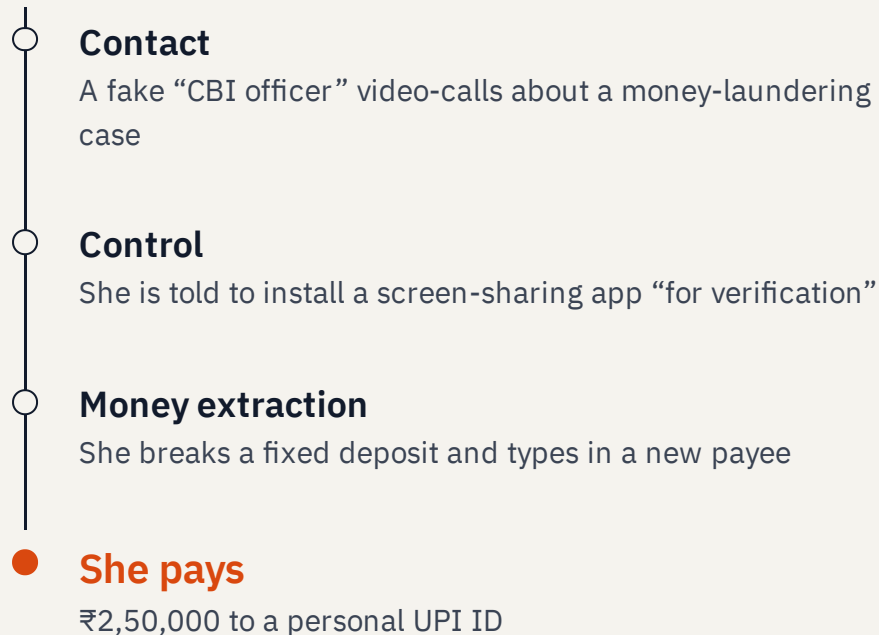
Mathews V Manoj B.Tech Electronics & Communication, Muthoot Institute of Technology and Science (KTU)

All results here come from synthetic data we generated.
No real customer, bank or platform data was used.

In these scams, the victim makes the payment

Most of the scam happens before any money moves. By the time the victim pays, the PIN is correct, the phone is their own and they intend to pay, so a check on the payment alone has little to go on.

A typical digital-arrest scam



A payment-level check does not see these steps

This is the only step it checks

₹22,845 cr

lost to cyber fraud in India in 2024

36.37 lakh

complaints filed in 2024

Source: Ministry of Home Affairs, reply in Lok Sabha, July 2025 (NCRP and CFCFRMS data).

Our idea: use the steps before the payment as evidence, not just the payment itself.

A scam has four stages. We look for all of them.

Usual approach

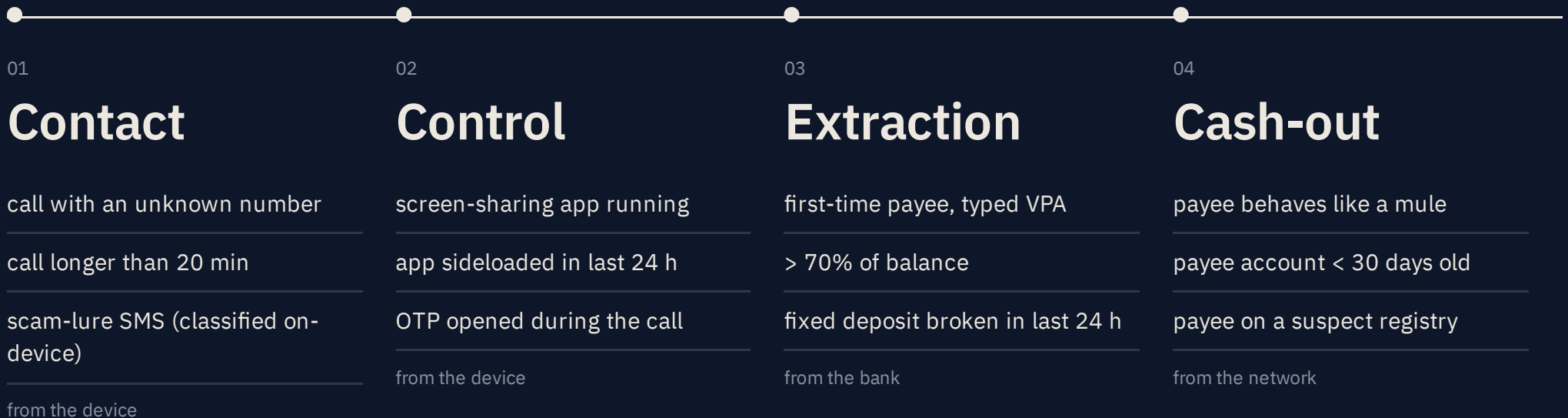
one payment

→

suspicious or not

Chakravyuh

contact → control → extraction → cash-out → response scaled to the evidence



One odd signal is not enough to stop a payment. **We act when several stages show up together.**

A genuine payment can look odd on its own, like a new payee on rent day or a long call with a plumber. A scam is easier to spot from the sequence. All 20 signals are listed in Appendix A.

Three levels of response

The more stages we see, the stronger the response. The bank chooses how many alerts per 1,000 payments it will accept at each level, and we set the thresholds to match that on separate test data.

Level 1

Log it

When

Score above τ_1

Customer sees

Nothing. The session goes to the bank's fraud team for later review.

20.4 genuine sessions per 1,000 reach this level
measured on synthetic data (target 20)

Level 2

Warn and ask

When

Score above τ_2 **and** ≥ 2 stages active

Customer sees

A warning that names the likely scam, and two questions. The customer can still pay.

2.77 genuine sessions per 1,000 reach this level
measured on synthetic data (target 3)

Level 3

Hold and review

When

Score above τ_3 **and** ≥ 3 stages, or 2 stages + registry hit

Customer sees

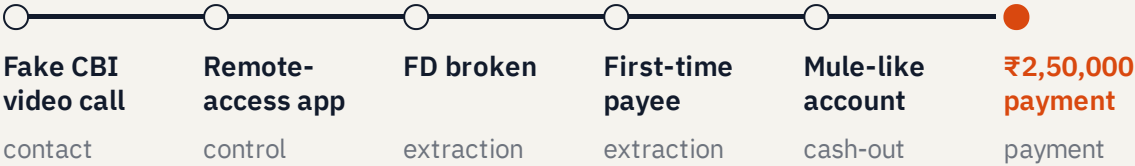
The payment is held for up to 30 minutes and a bank agent calls. Two staff must approve a release.

0.07 genuine sessions per 1,000 reach this level
measured on synthetic data (target 0.5)

Nothing is blocked permanently. A person decides every hold.

Example: Mrs R., 68

A made-up case based on reported digital-arrest scams.



Today

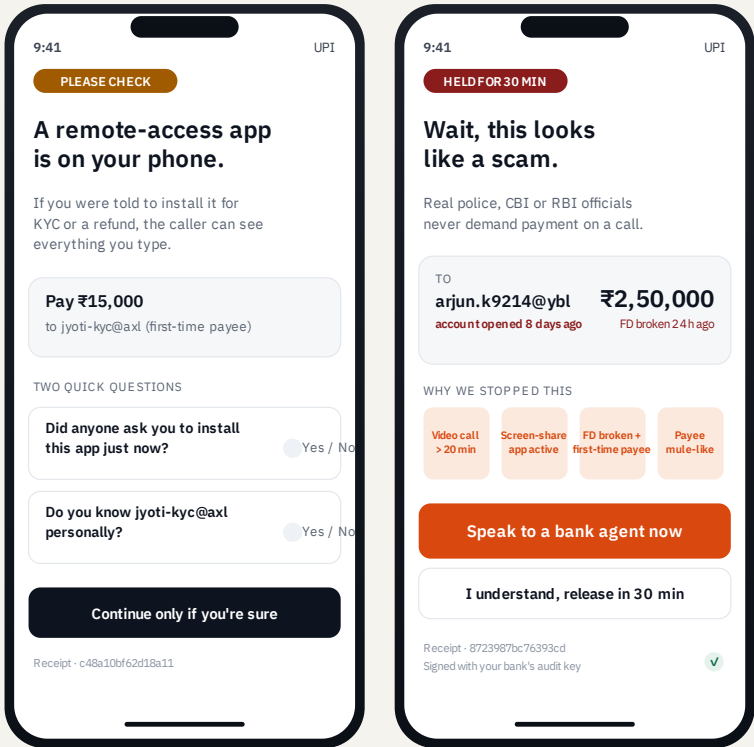
The money is gone.

A “new payee and high amount” rule shows a general warning. The caller tells her to ignore it, and she pays. The money goes to a mule account and is moved on within hours.

With Chakravyuh

The payment is held.

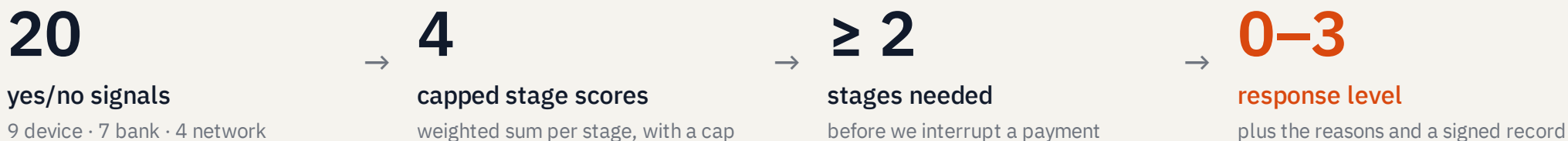
All four stages show up, so it goes to level 3. The screen tells her real officials never ask for money on a call, and a bank agent calls her before anything is released.



Level 2: warn and ask

Level 3: hold (her case)

How the score works



Score

$$s(x) = b + \sum_g \min(C_g, \sum_{k \text{ in } g} w_k x_k)$$

Decision

Level 2 if $s \geq \tau_2$ and active stages ≥ 2

Level 3 if $s \geq \tau_3$ and (active stages ≥ 3 or ≥ 2 plus a registry hit)

g is one of the four stages and x_k is 1 if signal k is present. The weights w_k come from a logistic regression (L2) on labelled sessions and are kept at zero or above. A stage counts as active when its capped score is at least 1.5. All values are in Appendix A.

Non-negative weights

A missing signal does not change the score, because a fraudster can always hide one.

Stage caps

A safety limit so one stage cannot dominate ($C = 5, 5, 6, 6$). It made no measurable difference on our data.

Two-stage rule

We only interrupt when at least two stages are active.

Prevalence adjustment

Training data is 3.8% scams; we adjust the model to an assumed 1 in 5,000 in real use.

Thresholds

τ is set on a separate dataset to meet the bank's alert limit, then fixed.

Results on synthetic data

| These numbers come from 8 synthetic test sets the model never saw. They are not results from a real bank.

92.3%

of scam sessions reach level 2 or 3 (recall)

95% confidence interval [91.7%, 93.0%]

2.77 / 1k

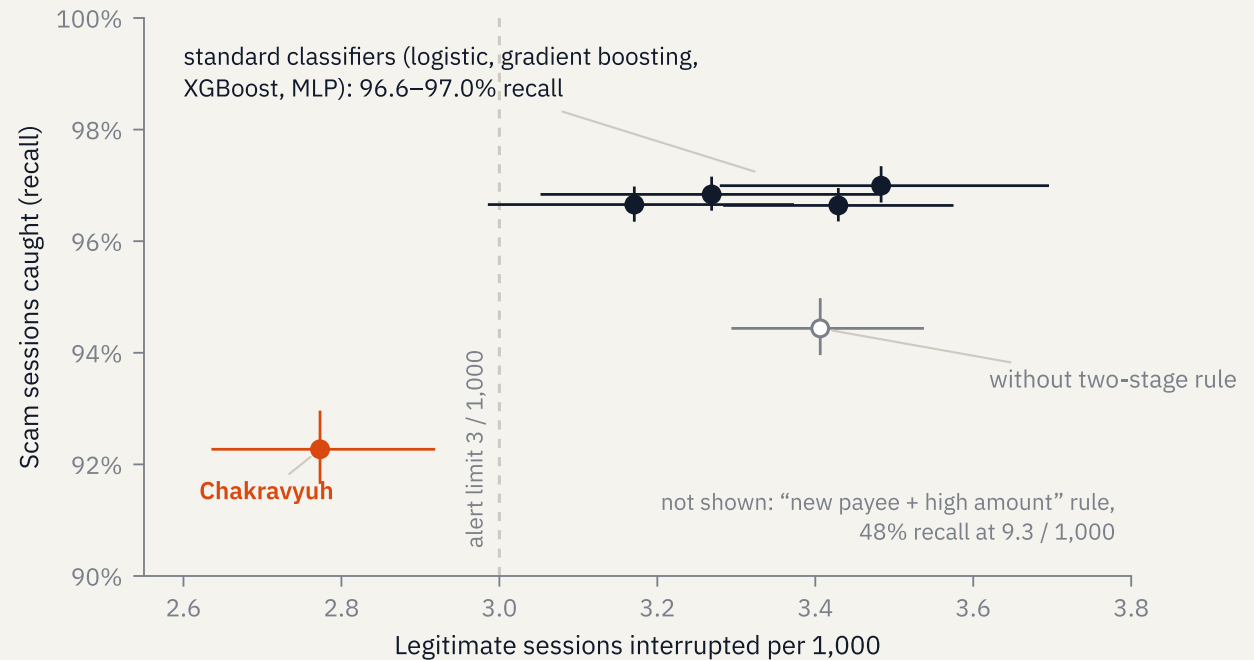
genuine sessions interrupted per 1,000

95% CI [2.64, 2.92], target 3.0

0.980

area under the precision-recall curve

95% CI [0.978, 0.982]



All models use the same training, threshold and test data. Bars show 95% confidence intervals.

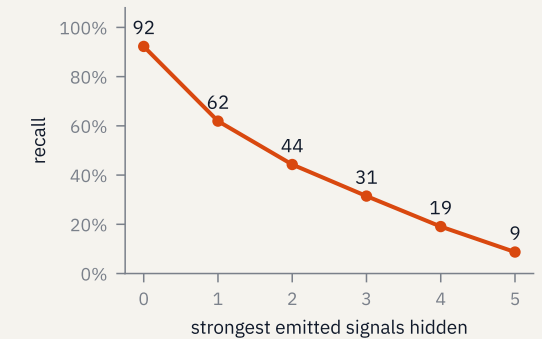
What we give up. Even when set to the same alert rate as ours, standard classifiers catch 96.1–96.8% of scams, about 4 points more. 47% of that gap is collect-request scams, which only ever show one stage. Without the two-stage rule our own score catches 94.4%. Slide 10 shows what the rule buys.

Trained on dataset 7, thresholds set on dataset 21, tested on datasets 101 to 108. Each has 60,000 genuine and 2,400 scam sessions. Scoring takes about 23 μ s per session on our machine.

What if the fraudster changes tactics?

Question	Our answer	What we found
Can they hide their tracks? Ask the victim to hang up before paying, skip screen-sharing, send less.	The payment and receiving-account signals come from the bank, which the caller cannot hide.	86.9% recall on scam sessions where the fraudster hides these signals (25% of scams). Worst case, hiding the strongest signal in each session (even ones a fraudster could not really hide): 62%. Two signals: 44%. measured
Can they split one payment into several? Four payments of ₹25,000 instead of one of ₹1,00,000.	A signal for two or more payments to new payees within 60 minutes.	Not separately simulated in Round 1. design only
Can they use older, rented mule accounts? An old account avoids the 'new account' signal.	The mule score also looks at how many new senders an account has and how fast money leaves it. Other stages still count.	The mule score catches only 24% of old mule accounts, but payments to them are still caught 73.6% of the time. measured
Can they poison the training data? Send lots of false 'scam' or 'genuine' reports.	Retrain only on cases the bank has confirmed. Customer reports alone are never used as labels.	Not evaluated. design only

Recall as more signals are hidden



We assume the fraudster has read our weights and removes the strongest signals first. Thresholds are not changed.

A scam type it has not seen

We trained without one scam type and tested on it: 79–93% recall.

Removing one stage

Without contact, control, extraction or cash-out signals, recall is 87%, 92%, 67%, 65%.

What data we use

Stays on the phone

- ✗ Call audio
- ✗ Screen frames
- ✗ Message text
- ✗ Contact list
- ✗ OTP values
- ✗ App inventory

phone

Sent to the bank

9 yes/no values, the amount and a hashed payee ID

A banking app can read 5 of them under Android and Play rules: call active and its length (phone-state permission), video/VoIP call (audio mode), a known screen-sharing app or accessibility service, an app drawing over our screen, and whether the payee was typed.

4 need restricted access, so we plan fallbacks: caller not in contacts (call-log access), scam SMS (SMS access), any app sideloaded (all-apps visibility), OTP opened in a call (other apps' notifications). Fallbacks: call length only, the bank's own OTP timing, and a fixed list of known remote-access apps.

Retrained with only the 5 readable signals: **91.5%** recall at 2.87 per 1,000 (synthetic), against 92.3% with all 9.

Already at the bank

7 signals the bank can work out from data it already has: first-time payee, amount compared with past payments, share of balance, a broken fixed deposit, recent small credits.

4 signals about the receiving account: mule score, account age, a suspect-registry match, and whether it is a collect request.

$9 + 7 + 4 = 20$ signals.

Signed decision record

```
{ log_id, ts, session, amount, evidence (true only), score, tier, playbook, active_stages,
  action } + HMAC-SHA256
```

Each decision gets one ID, so the customer, the bank agent and any complaint all point to the same evidence. **Already built:** the server accepts only the 20 known signal names and rejects repeated or more-than-15-second-old requests.

We followed the DPDP Act 2023 ideas of collecting only what is needed and using it for one purpose. This is a design choice, not a legal certification. The phone SDK is not built yet; the server side is.

Why the two-stage rule, and our plan for the finale

A plain logistic regression is also explainable and catches more scams. So what does the rule add?

Same alert rate, synthetic test data	Logistic regression	Chakravyuh
Scams caught	96.1%	92.3%
Genuine payments interrupted per 1,000	2.95	2.77
...of which showed only one stage	49%	0%

A genuine payment the rule spares (from our test data)

A shopkeeper pays ₹6,750 on a collect request from a new payee. Nothing else is unusual: no call, no screen-sharing, a normal receiving account. Logistic regression interrupts it. Chakravyuh sees one stage, so it only logs it.

The cost: scams that only ever show one stage, mostly collect-request scams. We plan a specific collect-request warning for those (finale step 3).

What we plan to build and show in the 48-hour finale

H+0-8

Replay harness

Run saved test sessions through the server and check the records.

H+8-20

Android signals

Read the 5 allowed phone signals on one test phone and compare with the synthetic values.

H+20-32

Collect-request rule

Add a warning for collect requests from individuals and re-test; then attack our own model.

H+32-44

End-to-end replay

Run 100 sessions through to agent review and release or report.

H+44-48

Security review

Walk through the records, replay checks, two-person release and known limits.

Built so far: scoring server, threshold setting, signed records with replay checks, synthetic data generator and evaluation, 8 automated tests. **Not built yet:** Android SDK, payment hold, agent screen.

System architecture

Solid boxes are built. Dashed boxes are simulated with synthetic data. Dotted boxes are planned but not built.

Phone (inside the bank's app)

Signal readers

simulated

Call state, audio mode, accessibility, known remote apps. 5 of 9 allowed; synthetic for now.

Request builder

planned

9 yes/no values, amount, payee hash, one-time nonce, timestamp, Play Integrity token.

Warning screens

simulated

Scam message, two questions, hold screen. Mock-ups and a web demo.

1. signals sent ↓ 5. response level, message and record ID sent back ↑

Bank fraud team servers

Scoring server

built

FastAPI /decide endpoint: stage scores, two-stage rule, level, reasons.

Input checks

built

Rejects repeated nonces, requests older than 15 s and unknown signal names.

Signed records

built

HMAC-SHA256 on every decision. Moving the key to a KMS is planned.

Agent screen

planned

Queue of level-3 holds with reasons; two staff to release.

2. payee data looked up ↑ 3. scored 4. level-3 hold sent to payments → 6. confirmed outcomes saved ↓

Bank data systems

Mule score

simulated

Number of new senders, how fast money leaves, account age. Synthetic 7-day graph.

Suspect list check

simulated

A yes/no flag, never shown to customers. Synthetic for now.

Weekly retraining

planned

Only bank-confirmed cases; tested in shadow mode first.

Payments and reporting

Payment hold

planned

A time-limited hold through the bank's payment system.

Reporting

planned

Agent reports confirmed scams to I4C (CFCFRMS / 1930).

built

simulated

planned

We have no integration with Amazon, NPCI, I4C or any bank yet.

Following one payment through the system

- 1

Customer taps Pay
The app sends the signals that are true, the amount, a payee hash, a nonce and a timestamp.
- 2

Check the request
The server rejects unknown signals, repeated nonces and anything older than 15 seconds.
- 3

Score
Payee data is added, then stage scores, the two-stage rule, the level and the top five reasons.
- 4

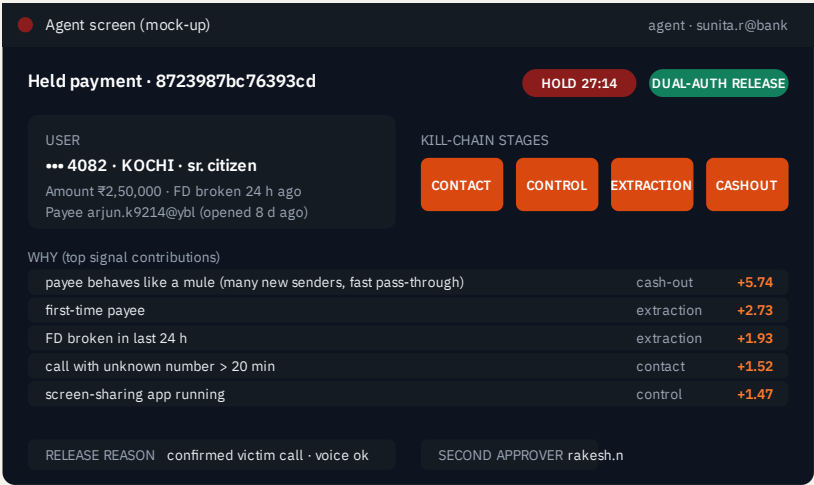
Respond
Level 0 allows, level 1 logs, level 2 warns and asks two questions, level 3 holds the payment.
- 5

Review
Level-3 cases go to a bank agent with the record and reasons. Two staff must approve a release.
- 6

Close
The payment is released, or the scam is confirmed and reported. The result is saved for retraining.

Security checks

Risk	What we do	
Old or repeated requests	One-time nonce and a 15-second limit	built
Extra or fake fields	Only the 20 known signal names are accepted	built
Edited decision records	Each record is signed with HMAC-SHA256	built
Fake or modified app	Play Integrity token tied to a hash of the request	planned
Staff misuse	Two approvers, both recorded in the signed record	planned
Record store leak	Records hold no raw content; identity is stored apart	design only
Poisoned training labels	Retrain only on bank-confirmed cases	design only



If a customer complains about a hold, they quote the record ID shown on their screen. The bank can then see exactly which signals caused it.

Rollout, speed and cost estimates

Rollout plan

Weeks 0 to 6: shadow	Score real payments but show nothing to customers. Measure the alert rate and reset the thresholds.
Weeks 6 to 10: levels 1 and 2	Customers who opt in, and large payments to individuals. Move on only if alerts stay within the limit.
From week 10: level 3	Payment holds, with agent review and two-person release.

Speed

Scoring takes about **23 μs** per session on our machine. We are aiming for under **40 ms** for the full round trip from the phone, but have not measured it.

What a pilot would measure

Alert rate on real payments, how many confirmed scams were caught, complaints, and agent review time, compared each week with the synthetic results here.

Estimates based on assumptions, not measured results

Possible savings (upper limit)

national reported loss, 2024 (MHA)	₹22,845 cr
× share for one pilot bank (our assumption)	2%
× recall on synthetic data	92.3%
= most that could be saved per year	₹422 cr

This assumes every level-2 or level-3 alert stops the whole loss. Some customers will pay anyway after a warning, so the real figure would be lower. It is an estimate, not a measured saving.

Cost for 30 million users, 250 payments each per year

servers (6 instances, peak about 2,378 requests/s)	₹4.2 lakh / yr
storing signed records (2.6 TB / yr)	₹0.6 lakh / yr
genuine payments held at level 3	≈ 1,370 / day
agent time at 5 minutes each	≈ 114 h / day

Assumptions: peak is 10 times the average load, 1,000 requests/s per server, doubled for backup, ₹8 per server-hour, 350 bytes per record, ₹2 per GB per month. Servers are cheap. **Staff time for reviews is the main cost**, so the bank should set the level-3 limit carefully. Development cost is not included.

Appendix A: signals and model details

Signal	Stage	Source	Weight	Meaning (as shown to customers and agents)
bait_credit_7d	contact	bank	+3.00	Small 'profit' credits from unknown senders in the last 7 days
call_long	contact	device	+1.52	That call has lasted more than 20 minutes
sms_scam_flag	contact	device	+1.50	A message received in the last 24h matches a known scam lure (KYC, parcel, electricity)
call_unknown_active	contact	device	+1.13	You are on a call with a number not in your contacts while paying
video_call	contact	device	+0.55	A video/VoIP call is active (common in 'digital arrest' scams)
sideload_24h	control	device	+2.32	An app was installed from outside the Play Store in the last 24h
remote_access	control	device	+1.47	A screen-sharing / remote-control app is running
accessibility_overlay	control	device	+1.34	A non-system app can read your screen and draw over the payment app
otp_read_in_call	control	device	+1.31	An OTP was opened while the unknown call was active
collect_from_p2p	extraction	network	+4.66	This is a 'collect' request from an individual (entering PIN SENDS money)
first_time_payee	extraction	bank	+2.73	You have never paid this account before
balance_drain	extraction	bank	+2.41	This payment empties more than 70% of your balance
vpa_typed	extraction	device	+2.08	Payee was typed or pasted instead of scanned or picked from contacts
fd_broken_24h	extraction	bank	+1.93	A fixed deposit was broken / loan taken in the last 24h
amount_high	extraction	bank	+1.71	Amount is far above your usual payments
staircase	extraction	bank	+1.19	Several payments to new accounts in the last 60 minutes
amount_very_high	extraction	bank	+0.51	Amount is extremely high for you (top 0.1% of your history)
payee_mule_high	cash-out	network	+5.74	The receiving account behaves like a mule account (many unrelated senders, money moved out fast)
payee_registry_hit	cash-out	network	+3.43	The receiving account is on the I4C / bank suspect registry
payee_new_account	cash-out	network	+1.00	The receiving account was opened less than 30 days ago

Model

$$s(x) = b + \sum_g \min(C_g, \sum_{k \in g} w_k x_k)$$

Weights: logistic regression with L2 ($C = 0.3$), trained on dataset 7 (60,000 genuine and 2,400 scam sessions), negative weights set to 0. A missing signal adds nothing.

The intercept b is adjusted from the training scam rate (3.85%) to an assumed real rate of 1 in 5,000.

Caps C : contact 5, control 5, extraction 6, cash-out 6. A stage is active when its capped score is 1.5 or more.

Thresholds (log-odds), set on dataset 21 to give 20, 3 and 0.5 genuine alerts per 1,000: $\tau_1 = -9.11$, $\tau_2 = -8.34$, $\tau_3 = -7.63$.

The scam type shown to the customer (digital arrest, remote-access KYC, investment or task scam, collect request) is picked by which signals are present. It changes the message, not the level.

Sources: 9 from the phone, 7 from the bank, 4 about the receiving account, 20 in total. We publish the weights on purpose; slide 8 tests a fraudster who has read them.

Appendix B: how we tested, and all results

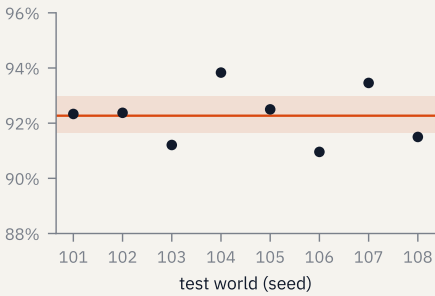
Data. Our generator (chakravyuh/synth.py) creates genuine sessions that include confusing cases (paying a plumber while on a call with him, rent day, ignored scam SMS) and four types of scam. In 25% of scam sessions the fraudster hides the call, video and screen-sharing signals and asks for less money. Payees come from a synthetic 7-day payment network with 120 mule accounts, a quarter of them over 120 days old. No real data is used.

Method. Train on dataset 7, set thresholds on dataset 21, test on datasets 101 to 108 (60,000 genuine and 2,400 scam sessions each). Every model uses the same data and a limit of 3 alerts per 1,000. Confidence intervals are bootstrapped over the 8 test sets.

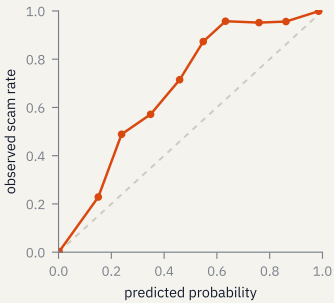
Model	Recall [95% CI]	Alerts per 1,000	AUPRC
Chakravyuh (tier ≥ 2)	92.3% [91.7%, 93.0%]	2.77 [2.64, 2.92]	0.980
Chakravyuh without two-stage rule	94.4% [94.0%, 95.0%]	3.41 [3.29, 3.54]	–
Logistic regression	96.6% [96.4%, 97.0%]	3.43 [3.28, 3.58]	0.984
Gradient boosting	96.7% [96.3%, 97.0%]	3.17 [2.99, 3.37]	0.985
XGBoost	96.8% [96.5%, 97.2%]	3.27 [3.05, 3.48]	0.985
MLP	97.0% [96.7%, 97.3%]	3.48 [3.28, 3.70]	0.985
Rule: new payee + high amount	48.1% [47.6%, 48.6%]	9.34 [9.10, 9.54]	–

The fixed rule has no threshold to tune, so it is shown as is.

Recall on each test set



Calibration



Calibration. Scores are set for a 1-in-5,000 scam rate, so for this check we shift them back to the test rate (3.85%). Brier score 0.0040, ECE 0.0024. Mid-range scores are a little under-confident.

Hidden signals (strongest first), number hidden and recall: 0: 92% · 1: 62% · 2: 44% · 3: 31% · 4: 19% · 5: 9%.

One stage removed (retrained without it): none: 92% (2.77/1k) · contact: 87% (2.98/1k) · control: 92% (3.50/1k) · extraction: 67% (3.10/1k) · cashout: 65% (2.37/1k).

Unseen scam type: digital arrest: 93% · remote access kyc: 89% · investment task: 81% · collect request: 79%. **By scam type:** digital arrest: 93% · remote access kyc: 97% · investment task: 92% · collect request: 83%.

By user type (recall / alerts per 1,000): student: 92% / 2.73 · shopkeeper: 92% / 2.64 · senior: 92% / 2.93 · professional: 93% / 2.79.

Stage caps made no difference on this data (same results without them). We keep them as a safety limit. Level-3 recall: 57.9%.

Appendix C: assumptions, regulations, code sketch, sources

Assumptions

Item	Value	Used in
Real scam rate	1 in 5,000 sessions	intercept adjustment
Alert limits	20, 3 and 0.5 per 1,000	set by the bank, per level
Fraudster hides signals	25% of scam sessions	synthetic data
Pilot bank share	2% of national reported loss	savings estimate
Load	30 M users × 250 payments/yr; peak 10× average	cost estimate
Unit costs	₹8 per server-hour; ₹2 per GB-month; 5 min per review	cost estimate

Regulations we designed around (not a legal certification)

DPDP Act 2023	Collect only what is needed: only yes/no values leave the phone.
RBI Master Direction on Digital Payment Security Controls (2021)	Fraud monitoring: step-by-step response with reasons and human review.
RBI Integrated Ombudsman Scheme (2021)	The record ID gives each complaint one reference to check.
PMLA 2002 record-keeping	Records kept for the required period, stored apart from identity data.
I4C CFCFRMS / 1930	Planned route for reporting confirmed scams. Not connected yet.

Sources

Ministry of Home Affairs, reply in Lok Sabha, July 2025: ₹22,845.73 crore lost to cyber fraud in 2024; 36.37 lakh complaints (NCRP and CFCFRMS). Digital Personal Data Protection Act, 2023. RBI Master Direction on Digital Payment Security Controls, 2021. RBI Integrated Ombudsman Scheme, 2021. I4C Suspect Registry, launched September 2024.

Android signal code (sketch, not built yet)

```
class ChakravvyuhProbes(ctx: Context, integrity: StandardIntegrityTokenProvider) {
    // Readable by a banking app (5 signals)
    fun evidence(nonce: String): Evidence {
        val ev = mapOf(
            "call_long" to (callActive() && callSeconds() >= 20 * 60), // TelephonyCallback, READ_PHONE_STATE
            "video_call" to (audio.mode == AudioManager.MODE_IN_COMMUNICATION),
            "remote_access" to (knownRemoteAppInstalled() // list of package names
                || a11yServicesEnabledByOthers()), // AccessibilityManager
            "accessibility_overlay" to lastTouchWasObscured(), // FLAG_WINDOW_IS_OBSCURED
            "vpa_typed" to payeeEnteredManually() // our own payment screen
        ).filterValues { it }
        // Restricted, not read: caller number (call log), SMS text, all installed apps,
        // other apps' notifications. Fallback: bank checks if its own OTP was sent during a call.
        val body = canonicalJson(ev, amount, payeeHash, nonce, now())
        return Evidence(body, integrity.request(requestHash = sha256(body)))
    }
}
```

Disclosures

All data is synthetic. We use no pre-trained models and no LLM in the decision. Code: Python (FastAPI, scikit-learn, XGBoost, NumPy, NetworkX, Matplotlib). Font: IBM Plex (SIL Open Font License). We used an AI assistant (Claude Code) to help write the code, text and layout. Every number comes from our evaluation script (eval/final_numbers.py). The phone and agent screens are mock-ups.